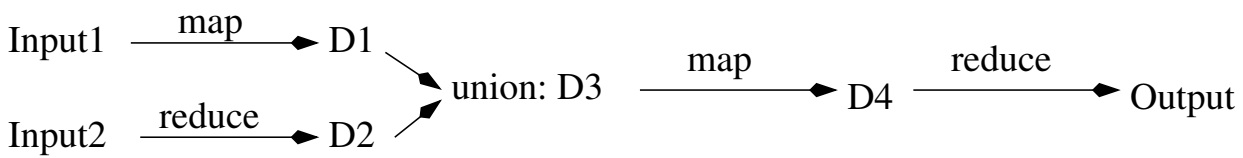
Spark framework

Baptiste Jeudy
baptiste.jeudy@univ-st-etienne.fr

Laboratoire Hubert Curien,
Université Jean-Monnet Saint-Etienne.

Master MLDM DSC

Spark Framework



- Extension of Map-Reduce
  - Several Map-like and reduce-like operations
  - Intermediate data not (always) stored on disk
    more efficient
  - Can be used with python, java, scala and R
  - Interactive mode with python: pyspark
- One main program (driver) executed on master node
- Several executor processes on worker nodes
  - Hold the distributed data
  - Do distributed computing
- Main web page:
  `http://spark.apache.org/`
- Documentation:
  `http://spark.apache.org/docs/latest/`

Spark Framework

- Several tools:
  - Spark SQL
  - MLlib for machine learning
  - GraphX for graph processing
  - ...
- We will focus on the core of the Spark framework:
  the extended map-reduce operations on distributed datasets

Resilient Distributed Datasets (RDDs)

- Driver process execute the program on master node
- Distributed computations managed through a "Spark Context" object
- Basic Data structure used for distributed computations:
- Resilient Distributed Datasets (RDDs):
  - Collection of objects (any Python object)
  - Distributed on worker nodes
  - Can be operated on in parallel
  - fault tolerant

RDD documentation:

`http://spark.apache.org/docs/latest/rdd-programming-guide.html`

RDD Creation

RDD can be created from:

- Python sequences (lists, tuples)
  - `sc.parallelize([1,4,6,2])`
  - Data sent from Driver node
- File on each worker node file system
- File on distributed file system like HDFS
  - Both with `sc.textFile("data.txt")`
- (`sc` is the Spark Context object)

Important parameter:

- Number of partitions the RDD is cut into
  - During computations: One task per partition
  - 2-4 partitions for each CPU
- Can be set manually (parameter) or automatically
  - `sc.parallelize(data, 10)`

Maps

- map can return any kind of object (even lists)
- if 2 map calls return <o1,o2,o3> and <o4,o5>
  What is the result of the map operation ?
  - a collection with 5 objects {o1, o2, ..., o5}?
  - a collection with 2 lists {<o1,o2,o3>, <o4,o5>}?
- use flatMap() for the first case
- use map() for the second case
- See example in the notebook

Reduction

- reduceByKey(f): apply function f for reduction
- Tree like reduction
  - reduction function f is $(V, V) \rightarrow V$
    (remark: key is not a parameter)
- f must be associative and commutative !
- Example: for sum, max (but not average)
- Does pre-reduce optimization on each node before the shuffle
  - less data send on the network

RDD Operations

Two kind of operations

- Transformation: RDD $\longrightarrow$ RDD
  - create a new RDD from an existing one
  - e.g., map like operations, some reduce operations
- Actions: RDD $\longrightarrow$ value
  - return a value to driver process
  - e.g., some reduce operations
  - Not to much data (otherwise memory pb for driver process)

Documentation for all RDD operations:

`http://spark.apache.org/docs/latest/rdd-programming-guide.html`

RDD Operations: Actions

Actions examples:

- collect(): return all the elements of the RDD to the driver program.
- count(): return the number of elements of the RDD.
- take(n): return the first n elements of the RDD.
- getNumPartitions(): number of partitions of the RDD
- reduce(f): aggregate all values using f.
  f must be of type $(V, V) \to V$ and associative and commutative.
  not the same as reduce in Map-Reduce framework!
- saveAsTextFile(path): save the RDD in a file.
- many others in the documentation

Return a value to the Driver process (except saveAsTextFile)

RDD Operations: Transformations

Transformations examples:

- map(f): apply f on each object of the RDD:
  new_RDD $= \{f(o)$ for all o in RDD$\} = \{f(o_1), f(o_2), ...\}$
- flatMap(f): new RDD is the concatenation of f results:
  new_RDD $= \bigcup_{\text{for all } o \text{ in RDD}} f(o) = f(o_1) \cup f(o_2) \cup ...$
- filter(f): select the objects of the RDD on which f return true:
  new_RDD $= \{o \in RDD$ such that $f(o)$ is true$\}$
- reduceByKey(f):
  f must be of type $(V, V) \rightarrow V$ and associative and commutative.
  On a RDD of (K, V) objects:
  - group by key K
  - then apply f on the values with same key
  - Return a new RDD of (K, V) objects
- groupByKey
- many others in the documentation

Create a new RDD from an existing one